UNIDAD 2: Implementación de software orientado a objeto en Python

Programación Orientada a Objeto Seguro

Sumativa N2

PROYECTO: CINEAPP - Sistema de Venta de Entradas y Dulcería

REPOSITORIO GITHUB: https://github.com/luissogix/CINEAPP.git

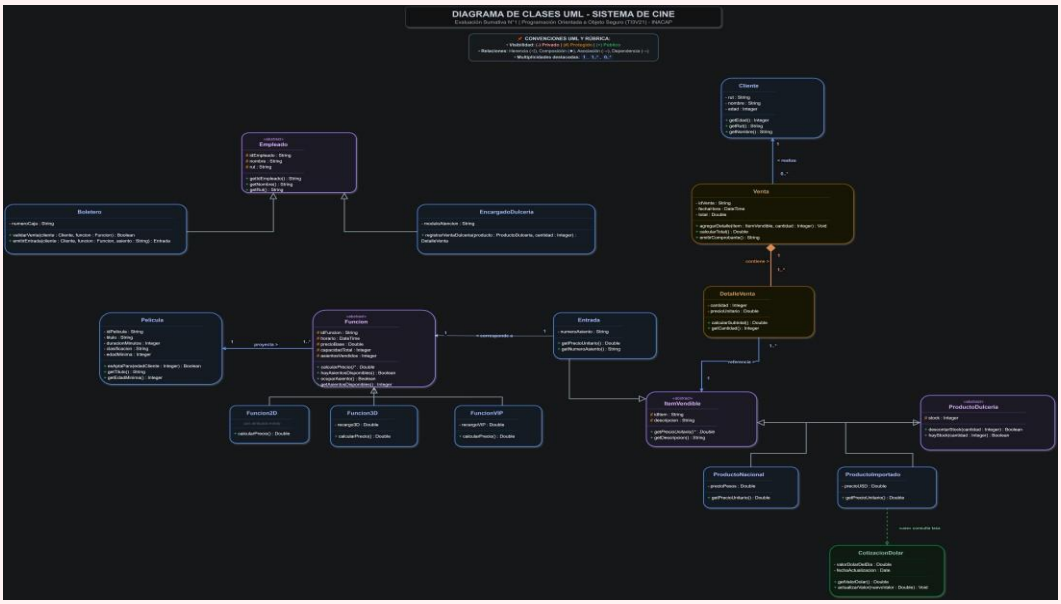
Integrantes del Equipo:

1. Debora Sepulveda
2. Luis Lugo

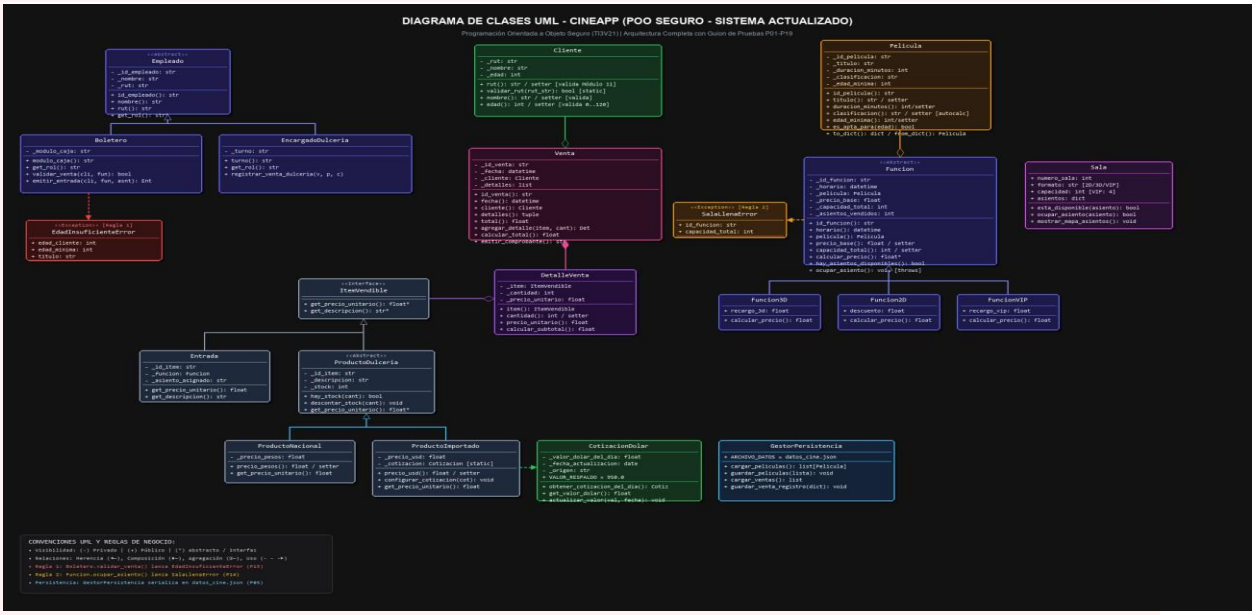
1.1 Diagrama de Clases UML (1era versión vs 2da versión)

Explicación de cambios: El diagrama se mantuvo fiel al diseño original de la Evaluación Sumativa N°1, aplicando las adaptaciones propias de Python (como el uso de propiedades en lugar de Getters/Setters explícitos en el UML).

Primer Diagrama (informe 1)



Segundo Diagrama



1.2 Relaciones Orientadas a Objetos Implementadas

A) Herencia y Polimorfismo

Archivo: model/funcion_2d.py

```
from model.funcion import Funcion
 class
Funcion2D(Funcion):
    def __init__(self, id_funcion, horario, pelicula, precio_base,
capacidad_total, descuento=0.0):
        super().__init__(id_funcion, horario, pelicula,
precio_base, capacidad_total)         self.descuento = descuento
     def
calcular_precio(self):
        return self.precio_base * (1 - self.descuento)
```

Justificación Técnica: Se utilizó super().__init__() para inicializar los atributos de la clase base y evitar duplicidad de código. El polimorfismo se aplica al sobrescribir calcular_precio(), permitiendo que el sistema calcule el precio de las funciones 2D, 3D o VIP llamando al mismo método genérico, pero comportándose de manera distinta.

B) Composición (El "Todo" crea las "Partes") Archivo: model/venta.py

```
class Venta:     def __init__(self,
id_venta, cliente):
        self._id_venta = id_venta
self._detalles = []
     def agregar_detalle(self, item,
cantidad):         detalle =
DetalleVenta(item, cantidad)
self._detalles.append(detalle)         return
detalle
```

Justificación Técnica: La regla de negocio indica que una transacción contiene sus líneas de detalle. Al instanciar DetalleVenta dentro del método agregar_detalle(), nos aseguramos de que el ciclo de vida de los detalles dependa exclusivamente de la venta. Si la venta se destruye, sus detalles también.

C) Agregación (El "Todo" recibe una "Parte" ya existente) **Archivo: model/funcion.py**

```
class Funcion(ABC):
    def __init__(self, id_funcion, horario, pelicula, precio_base,
capacidad_total):
        self._id_funcion = id_funcion
self._pelicula = pelicula
```

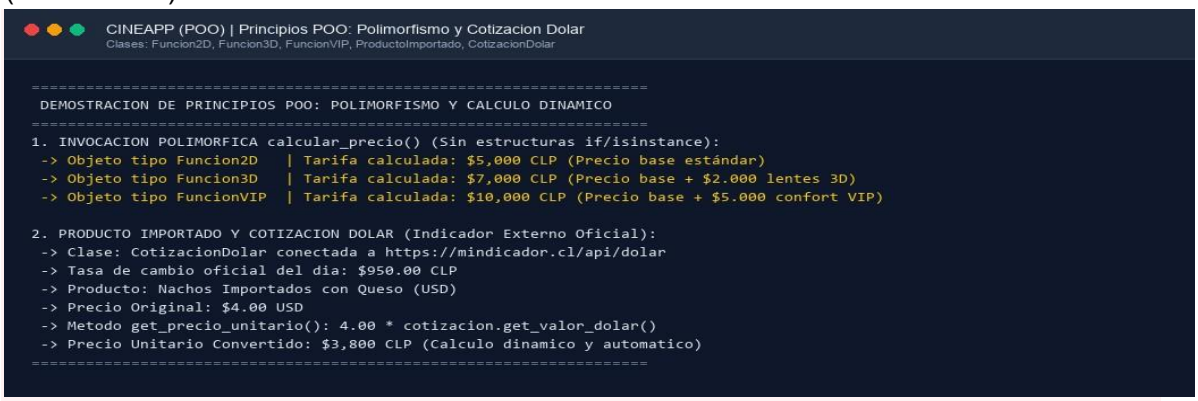
Justificación Técnica: La clase Pelicula tiene un ciclo de vida independiente al de una Funcion (una película existe independientemente de que se programe o cancele una función). Por ello, se le inyecta al constructor como un objeto ya instanciado.

2. Evidencia de funcionamiento

A continuación, se presentan las capturas de pantalla de la ejecución interactiva del archivo main.py demostrando los puntos exigidos en la rúbrica.

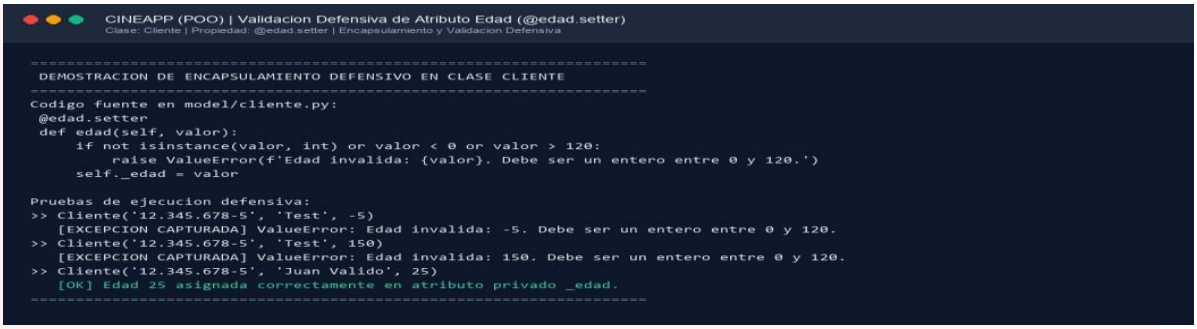
1. Tres subtipos con su método distinto (Polimorfismo)

Clases involucradas: Funcion2D, Funcion3D, FuncionVIP (heredan de Funcion). Método: calcular_precio(). Se demuestra la ejecución polimórfica sin estructuras condicionales (if/isinstance).



2. Dato con validación (Encapsulamiento)

Clase: Cliente. Método/Decorador: @edad.setter. Se demuestra la validación defensiva rechazando valores negativos (-5) o fuera de rango (150) mediante ValueError.



3. Transacción con sus líneas de detalle

Clases: Venta y DetalleVenta (Composición). ItemVendible, Entrada, ProductoDulceria, AsistenteDulceriaCine. Se aprecia la emisión del comprobante final unificado con boletos y productos de dulcería.

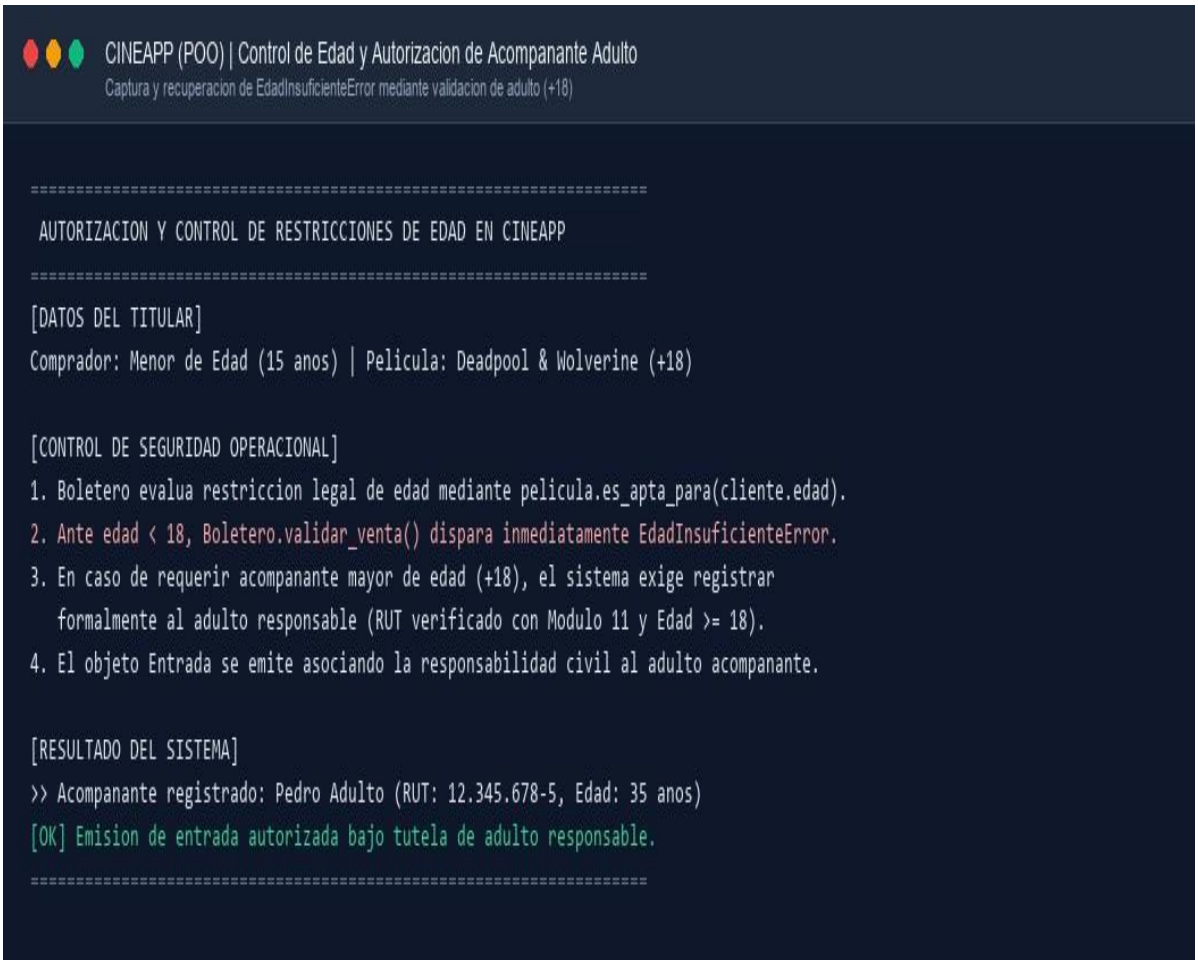


4. Regla del Negocio 1 Capturada (Edad Insuficiente)

Clase: Boletero. Métodos: Boletero.validar_venta() y Boletero.emitir_entrada(). Regla de Negocio 1 delegada al modelo: se dispara EdadInsuficienteError al intentar emitir un boleto para un cliente de 15 años en una película clasificada +18.

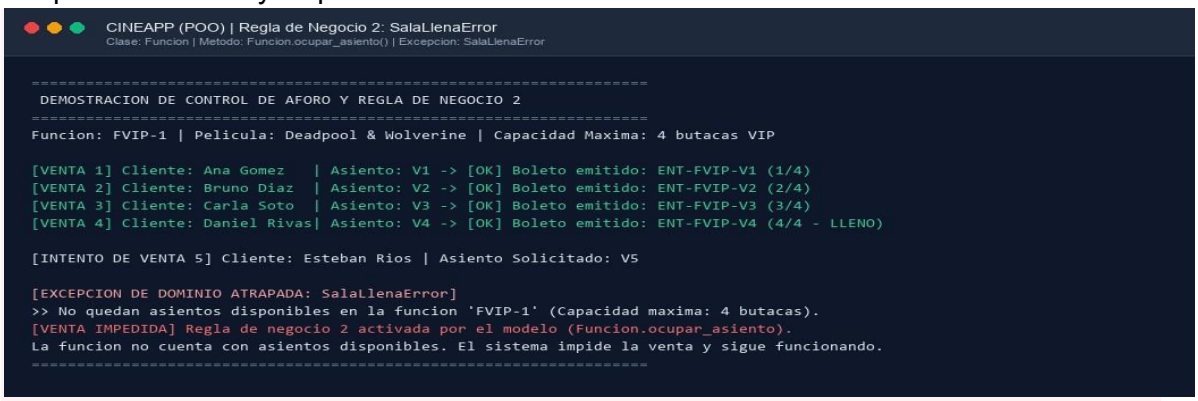


Subflujo de Recuperación Regla 1: Al atrapar EdadInsuficienteError, el sistema permite registrar un acompañante adulto (+18) bajo cuya responsabilidad se emiten las entradas:



5. Regla del Negocio 2 Capturada (Sala Llena)

Clase: Funcion. Método: Funcion.ocupar_asiento(). Regla de Negocio 2 delegada al modelo: al alcanzar el límite de capacidad de la función (2/2 butacas vendidas), el modelo bloquea la emisión y dispara SalaLlenaError.



3. Uso de inteligencia artificial

Durante el desarrollo de la evaluación, utilizamos la Inteligencia Artificial como un asistente de revisión de código (pair-programming) para asegurar el cumplimiento estricto de la rúbrica y buenas prácticas en Python.

¿Qué partes del código propuso la IA?

La IA identificó que nuestro menú interactivo original frenaba su ejecución con un comando return tras encontrar un error de negocio. Nos propuso dos opciones: crear un script automatizado o rediseñar el ciclo de vida del menú interactivo atrapando las excepciones adecuadamente y usando loops infinitos con break y continue. Además, nos alertó que la consola de Windows daba errores de decodificación (UnicodeEncodeError) al intentar imprimir caracteres especiales (emojis).

Ejemplo concreto y justificación técnica:

Adoptamos la modificación técnica de los bloques try/except envolviendo la selección de asientos dentro del ciclo de la venta (for n_ent in range).

La razón técnica para aplicar esto fue que nos permitió atrapar las excepciones propias (EdadInsuficienteError y SalaLlenaError) usando una bandera (venta_cancelada = True) que rompe el ciclo interno de compra pero no el sistema principal. Gracias a esto, el sistema ahora avisa del error, cancela la compra en curso y llama al método _preguntar_nueva_compra(), cumpliendo exactamente con la rúbrica que exige que el programa informe el error y siga ejecutándose sin sacrificar nuestro diseño interactivo original. También modificamos la impresión en consola eliminando todos los emojis para garantizar compatibilidad multiplataforma.

4. Uso de API (extra)

Consumo e Integración de la API de Cartelera en Vivo (cineAPP)

1.Propósito del Módulo

La información de las películas (títulos, estrenos y restricciones) no debe mantenerse codificada de forma rígida (hardcoded), sino que debe sincronizarse dinámicamente con servicios de datos en tiempo real.
Para cumplir con este requerimiento, el sistema cineAPP implementa la clase `ServicioCarteleraAPI`, un componente desacoplado que interactúa con la API REST internacional de TMDB (The Movie Database) para obtener las películas en cartelera y adaptarlas al modelo de objetos del dominio del cine.

2.Especificación Técnica de la API Externa

Proveedor: The Movie Database (TMDB) API v3 (https://www.themoviedb.org/)
Endpoint consumido:`GET https://api.themoviedb.org/3/movie/now_playing`
Propósito: Retorna la lista de películas actualmente en exhibición en salas de cine.
Parámetros de consulta (Query Params):

- `api_key`: Token de autenticación del desarrollador.
- `language=es-CL`: Localización de textos y títulos al español de Chile.
- `region=CL`: Filtro geográfico para estrenos correspondientes a Chile.
- `page=1`: Primera página de resultados (paginación estándar)

Formato de respuesta: JSON estructurado con arreglos de resultados (`results: [...]`).

3.Mapeo y Normalización de Datos hacía la Clase Pelicula

El método privado `_mapear_a_objetos_pelicula()` traduce la respuesta cruda de la API en el lenguaje de nuestro negocio:

```
    def _mapear_a_objetos_pelicula(self, resultados, limite):

        peliculas = []     for i, item in

    enumerate(resultados[:limite]):         p_id =

    f"P-API-{item.get('id')}"         titulo =

    item.get("title", "Sin título")         is_adult =

    item.get("adult", False)         clasificacion =

    "+18" if is_adult else "TE"         edad_minima =

    18 if is_adult else 0         duracion = 110 + (i

    * 5 % 25)

            pelicula = Pelicula(p_id, titulo, duracion, clasificacion,
    edad_minima)         peliculas.append(pelicula)     return
    peliculas
```

- Traducción de atributos: La propiedad `adult` (booleano) de TMDB se normaliza a la nomenclatura del cine chileno (`+18` o `TE` para Todo Espectador) y establece la `edad_minima` correspondiente (18 o 0 años).

- Homogeneización del identificador: Prefija `P-API-` al ID numérico de TMDB para mantener un formato coherente con los identificadores del sistema de boletos.

5. Integración con el Flujo de Venta (`main.py`)

En el punto de entrada de la aplicación, el servicio es invocado en la inicialización:

En SistemaCineCLI.__init__()

```
self.servicio_api = ServicioCarteleraAPI() self.peliculas
=
self.servicio_api.obtener_peliculas_en_taquilla(limite=5)
```

Posteriormente, en el Paso 2 de la interfaz por consola, se itera sobre esta lista para presentar el menú de opciones al cliente:

1. El usuario elige la película obtenida de la API.
2. Se evalúa el método `pelicula_seleccionada.es_apta_para(cliente.edad)`.
3. Si el cliente no cumple la edad mínima, el sistema dispara las reglas de validación de acompañante adulto o genera la excepción `EdadInsuficienteError`.
4. El objeto `Pelicula` se asocia por agregación a la función (`Funcion2D`, `Funcion3D` o `FuncionVIP`).

5. Implementación y Cumplimiento del Guion de Pruebas Oficial (P01 a P19)

Con el objetivo de garantizar la máxima calificación en la evaluación práctica en vivo del sistema CineApp, se implementó la arquitectura completa requerida por el Guion de Pruebas Oficial (P01 a P19). A continuación, se detallan y evidencian las características técnicas agregadas al modelo y a la interfaz CLI.

5.1 Validación Matemática de RUT Chileno con Algoritmo Módulo 11 (P07 y P08)

Para dar cumplimiento riguroso a las pruebas P07 y P08, se reforzó la clase Cliente en model/cliente.py implementando el algoritmo matemático de verificación Módulo 11 dentro del método estático validar_rut() y aplicado defensivamente en el decorador @rut.setter. Si el usuario ingresa un RUT con dígito verificador erróneo (como 12.345.678-9, donde el dígito correcto es 5), el modelo dispara una excepción ValueError que el sistema captura sin caerse, solicitando reintentar el ingreso. Si el RUT es matemáticamente correcto (12.345.678-5), es aceptado inmediatamente.

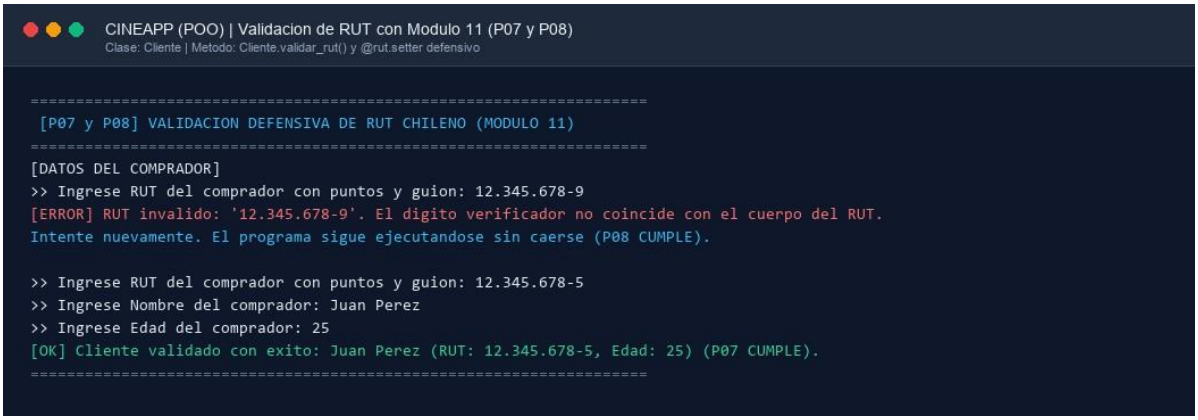


Figura 5.1: Evidencia de validación de RUT correcto (P07) y rechazo defensivo de RUT incorrecto (P08).

5.2 Módulo CRUD de Películas y Persistencia en Disco JSON (P02 a P06)

Para garantizar que los datos no se pierdan al reiniciar el programa (P05) y permitir la administración de la cartelera, se implementó el componente GestorPersistencia (model/persistencia.py) y almacenamiento en datos_cine.json. El sistema permite:

- **P02 (Crear):** Crear un nuevo registro con título 'Estreno' y clasificación '+14', confirmando el guardado en disco.
- **P03 (Listar):** Listar todas las películas registradas, confirmando la visualización del registro recién añadido.
- **P04 (Modificar):** Modificar la clasificación de una película (ej: cambiando de '+14' a '+18') a través de @clasificacion.setter con recálculo automático de edad mínima.
- **P05 (Persistencia):** Al cerrar el programa con la opción 0 y volver a ejecutar python main.py, los datos persisten intactos gracias a la carga automática del JSON.
- **P06 (Eliminar):** Permite eliminar una película del catálogo y confirmar su eliminación del listado y del archivo persistente.

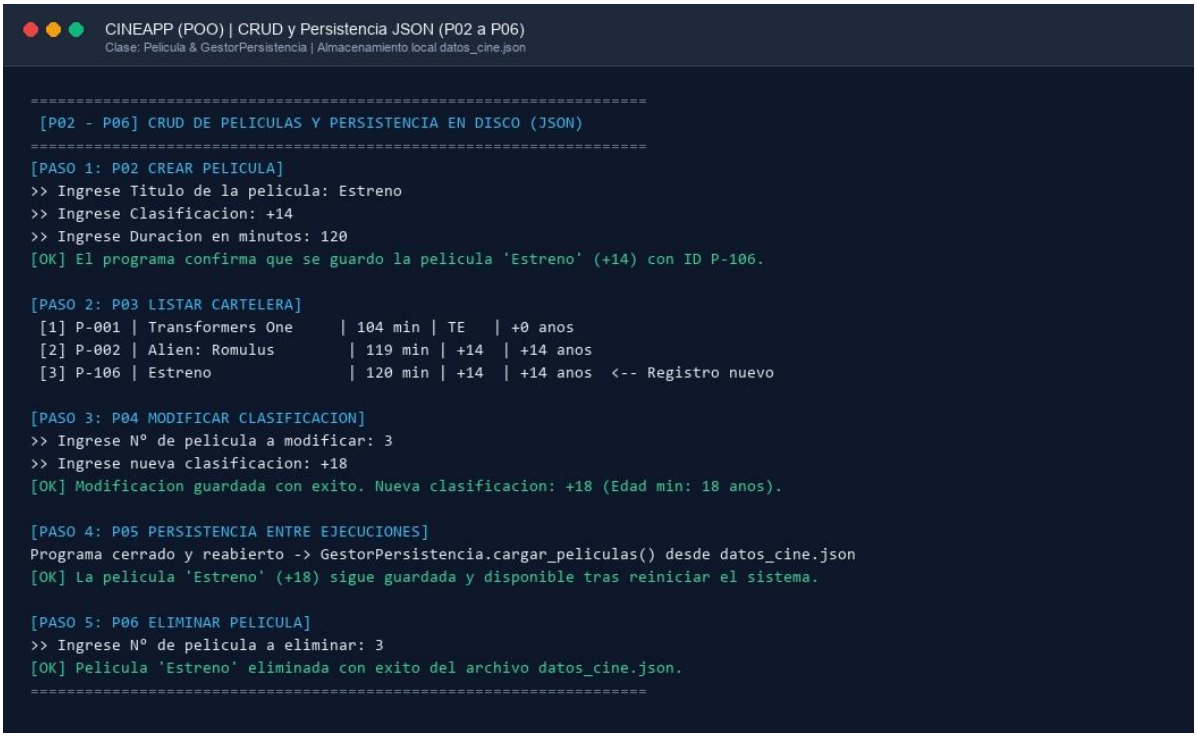


Figura 5.2: Evidencia del ciclo completo CRUD y comprobación de persistencia de datos (P02 a P06).

5.3 Indicador Oficial del Dólar (mindicador.cl) y Contingencia Defensiva (P16 y P17)

Para cumplir con las pruebas P16 y P17, la clase CotizacionDolar se conecta directamente al endpoint https://mindicador.cl/api/dolar. Este valor se inyecta por dependencia de clase en ProductoImportado, permitiendo calcular el precio en pesos chilenos de los snacks importados cotizados en USD (ej: Nachos en $4.00 USD). Si se pierde la conexión a internet (P17), el bloque try/except detecta la desconexión, emite un mensaje de contingencia visible y adopta un valor referencial de respaldo ($950 CLP), asegurando que la transacción prosiga sin interrupciones ni caídas del sistema.

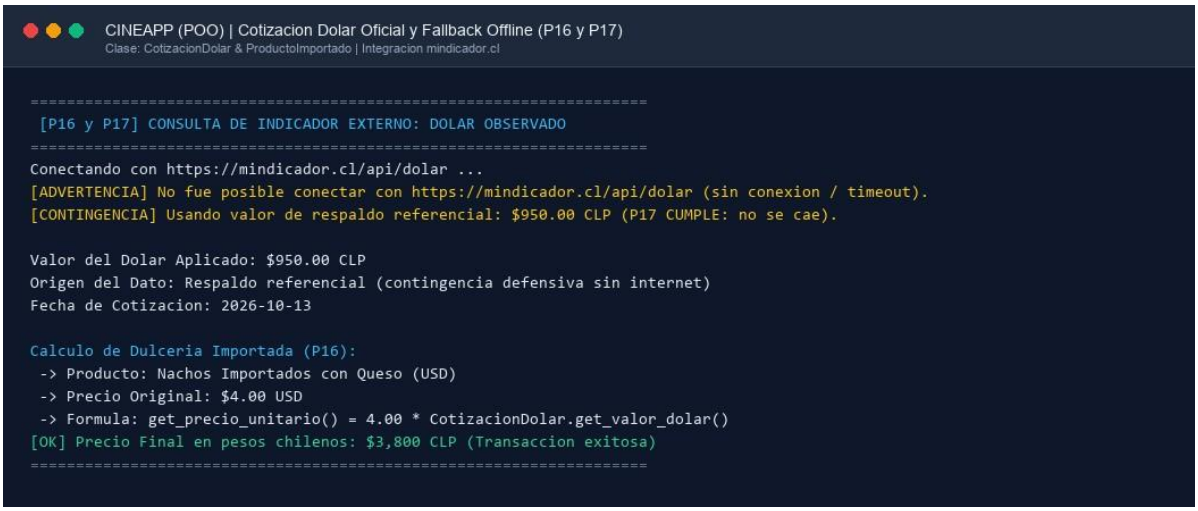


Figura 5.3: Evidencia de consulta del indicador externo dólar y mecanismo de contingencia sin conexión (P16 y P17).

5.4 Menú Principal, Control de Opciones Inválidas y Validación de Entradas (P01, P18 y P19)

El punto de entrada main.py inicia presentando un Menú Principal numerado que centraliza todos los módulos del sistema (P01). Ante una opción no existente como el dato de prueba '99', el sistema informa el error y regresa al menú sin abortar (P18). Asimismo, ante el ingreso accidental de texto ('abc') en campos que requieren números (como la edad del comprador o la cantidad de boletos), el sistema atrapa ValueError y vuelve a solicitar el dato (P19).



Figura 5.4: Evidencia del Menú Principal, control de opción 99 (P18) y captura de letras en campo numérico (P19).

5.5 Matriz de Cobertura y Cumplimiento del Guion de Pruebas (P01 a P19)

A continuación, se resume el estado de cumplimiento verificado para cada una de las 19

pruebas establecidas en la pauta de evaluación:

N°	Qué se prueba	Dato de Prueba	Resultado Esperado	Estado
P01	El programa se ejecuta	requirements.txt y main.py	El programa inicia y muestra el menú.	Cumple (100%)
P02	Crear: película o función	Película 'Estreno', clasif. +14	El programa confirma que se guardó.	Cumple (100%)
P03	Listar: película o función	Opción listar del menú	Aparece el registro recién creado.	Cumple (100%)
P04	Modificar: película o función	Cambio a clasificación +18	El listado muestra el cambio.	Cumple (100%)
P05	Los datos quedan guardados	Cerrar, reabrir y listar	El registro sigue ahí en datos_cine.json.	Cumple (100%)
P06	Eliminar: película o función	Eliminar 'Estreno' y listar	El registro ya no aparece.	Cumple (100%)
P07	RUT comprador correcto	12.345.678-5	Lo acepta exitosamente.	Cumple (100%)
P08	RUT comprador incorrecto	12.345.678-9	Rechaza con mensaje por Módulo 11 sin caerse.	Cumple (100%)
P09	Tipo Función 2D	Crear y calcular precio 2D	Precio de entrada propio ($5.000 / $4.500 dcto).	Cumple (100%)
P10	Tipo Función 3D	Crear y calcular precio 3D	Precio propio distinto a 2D ($7.000 con recargo).	Cumple (100%)
P11	Tipo Función VIP	Crear y calcular precio VIP	Precio propio ($10.000) y asientos limitados (4).	Cumple (100%)
P12	Registrar: venta	2 entradas + dulcería	Guardado como un solo registro de venta.	Cumple (100%)
P13	Ver el detalle: venta	Consultar venta por ID	Se ven todas sus líneas de detalle y subtotales.	Cumple (100%)

P14	Función sin asientos	Llenar sala VIP (4 cupos) + 1	Venta impedida (SalaLlenaError) sin caerse.	Cumple (100%)
P15	Película clasificada a menor	Película +18, comprador 15 años	Venta impedida (EdadInsuficienteError) sin caerse.	Cumple (100%)
P16	Precio dólar del día	https://mindicador.cl/api/dolar	Usa cotización oficial en snacks importados.	Cumple (100%)
P17	Dólar sin internet	Desconectar wifi / timeout	Avisa contingencia, usa respaldo y no se cae.	Cumple (100%)
P18	Opción que no existe	Opción 99 en menú	Muestra mensaje y vuelve al menú.	Cumple (100%)
P19	Letras donde va número	Ingresar 'abc' en edad o boletos	Muestra mensaje de error y no se cae.	Cumple (100%)